

**FAST NATIONAL UNIVERSITY
OF COMPUTER AND EMERGING SCIENCES**

Project: HOSPITAL MANAGEMENT SYSTEM

Programming Fundamentals

Spring 2026

Submitted By:

Member 1: Bisma Khan **Roll no.:** 25I-6581

Member 2: Bismah Mamoon **Roll no.:** 25I-6550

Submitted To:

Course Instructor: Engr. Sana Saleh

Lab Instructor: Engr. Alina Gulzar

Department of Computer Engineering

Submission Date:

10th May, 2026

PROJECT REPORT

Introduction

The Hospital Management System is a menu-driven C++ application developed to manage basic hospital operations efficiently. This project is based on Programming Fundamentals concepts such as structures, functions, arrays, dynamic memory allocation, file handling, loops, conditional statements, searching, and sorting techniques.

The system manages important hospital records including patients, doctors, appointments, and treatments. All records are stored in text files so that data remains saved even after the program is closed.

The project contains different modules for hospital management. The Patient Management module allows adding, updating, deleting, searching, and viewing patient records. The Doctor Management module stores doctor details such as specialty and years of experience. The Appointment Management module schedules appointments between patients and doctors while checking slot availability and suggesting the next available slot if a requested time is already booked. The Treatment and Billing module stores treatment records, updates payment status, and generates bills for patients.

The system also performs automatic file cleaning and validation at startup. Invalid records such as incorrect phone numbers, duplicate IDs, empty names, or incomplete data are automatically removed to maintain accurate records.

Dynamic arrays are used to handle records efficiently whenever data is added or deleted. File handling is implemented using text files including patients.txt, doctors.txt, appointments.txt, and treatments.txt.

This project helped in understanding the practical implementation of Programming Fundamentals concepts and demonstrated how hospital operations can be managed through programming.

CODE

```
/*
 * =====
 *   HOSPITAL MANAGEMENT SYSTEM
 *   Programming Fundamentals -
 *   Bisma Mamoon And Bisma Khan
 * =====
 */

#include <iostream>
#include <fstream>
#include <string>
#include <cstring>
#include <cctype>
#include <iomanip>

using namespace std;

int findPatientIndex(int id);
int findDoctorIndex(int id);

// _____

//   STRUCTURES

// _____

struct Patient
{
```

```
    int    patientId;

    string name;

    int    age;

    string gender;

    string contact;

    double balance;
};

struct Doctor
{
    int    doc_id;

    string name;

    string specialty;

    int    experience;
};

struct Appointment
{
    int    patientId;

    int    doctorId;

    char date[11];    // MM-DD-YYYY

    char time[10];    // hh:mm AM/PM
};
```

```
struct Treatment
{
    int    patientId;
    string description;
    double cost;
    bool   paid;
};

// -----
//  GLOBAL DYNAMIC ARRAYS
// -----

Patient*    patients      = nullptr;
Doctor*     doctors       = nullptr;
Appointment* appointments = nullptr;
Treatment*  treatments    = nullptr;

int patientCount    = 0;
int doctorCount     = 0;
int appointmentCount = 0;
int treatmentCount  = 0;

// -----
//  EMPLOYEE CREDENTIALS
```

```
// _____  
  
const string EMP_ID    = "123";  
  
const string EMP_PASS = "pass123";  
  
  
// _____  
  
//  UTILITY / VALIDATION HELPERS  
  
// _____  
  
  
// Trim leading/trailing whitespace  
string trim(string s)  
{  
    int start = 0;  
  
    int end = s.length() - 1;  
  
    // Find first non-space  
    while (start < s.length() && s[start] == ' ')  
    {  
        start++;  
    }  
  
    // Find last non-space  
    while (s[end] == ' ')  
    {  
        end--;  
    }  
  
    string result = "";
```

```
    for (int i=start;i<=end;i++)
    {
        result+= s[i];
    }
    return result;
}

// Check if string contains only digits
bool allDigits(string s)
{
    if (s == "")
        return false;

    for (int i = 0; i < s.length(); i++)
    {
        if (s[i]< '0' || s[i] > '9')
            return false;
    }

    return true;
}

// Validate phone: must be 11 digits, all numeric
```

```

bool isValidPhone(const string& phone)
{
    string p = trim(phone);
    if (p.length() != 11)
        return false;
    return allDigits(p);
}

int safeStoi(const string& s, int def = 0) {
    string t = trim(s);
    if (t.empty()) return def;
    try { return stoi(t); } catch (...) { return def; }
}

double safeStod(const string& s, double def = 0.0) {
    string t = trim(s);
    if (t.empty()) return def;
    try { return stod(t); } catch (...) { return def; }
}

// Convert "M" -> "Male", "F" -> "Female"
string normalizeGender(const string& g)
{
    string s = trim(g);
    if (s == "M" || s == "m")

```

```

        return "Male";

    if (s == "F" || s == "f")

        return "Female";

    return s;    // already "Male" / "Female"
}

// Convert 24h time string to 12h AM/PM e.g. "13:25" -> "01:25 PM"
string convertTo12Hour(string time)
{
    int hour;

    string minutes;

    string period;

    hour = (time[0] - '0') * 10 + (time[1] - '0');

    minutes = "";

    minutes += time[3];

    minutes += time[4];

    if (hour == 0)
    {
        hour = 12;

        period="AM";
    }

    else if (hour < 12)

```

```
{  
    period ="AM";  
}  
else if (hour == 12)  
{  
    period ="PM";  
}  
else  
{  
    hour =hour - 12;  
    period = "PM";  
}  
  
string result ="";  
  
if (hour < 10)  
    result += '0';  
  
result += to_string(hour);  
result += ":";  
result += minutes;  
result += " ";  
result += period;
```

```
        return result;
    }

    // Convert date from YYYY-MM-DD to MM-DD-YYYY
    string normalizeDateFormat(string date)
    {
        string first = "";
        string second = "";
        string third = "";
        int i = 0;

        // First part
        while (date[i] != '-')
        {
            first += date[i];
            i++;
        }
        i++;

        // Second part
        while (date[i] != '-')
        {
            second += date[i];
            i++;
        }
    }
}
```

```

    i++;

    // Third part
    while (i < date.length())
    {
        third += date[i];
        i++;
    }

    // If first part length is 4 → YYYY-MM-DD
    if (first.length() == 4)
    {
        return second + "-" + third + "-" + first;
    }

    // Otherwise assume DD-MM-YYYY
    else
    {
        return second + "-" + first + "-" + third;
    }
}

// Check a patient record has the minimum required fields
bool isValidPatientRecord(const Patient& p)
{
    if (p.patientId <= 0)
        return false;
}

```

```
        if (trim(p.name).empty())
            return false;

        if (!isValidPhone(p.contact))
            return false;

        return true;
    }

// _____
//  LOAD / SAVE FILES
//  _____

void loadPatients()
{
    ifstream fin("patients.txt");

    if (!fin)
    {
        return;
    }

    Patient temp[1000];

    int count = 0;

    while (!fin.eof())
    {
        Patient p;
```

```
string data;

// Patient ID

getline(fin, data, '#');

if (data == "")
{
    break;
}

p.patientId = stoi(data);


// Name

getline(fin, p.name, '#');


// Age

getline(fin, data, '#');

p.age = safeStoi(data);


// Gender

getline(fin, p.gender, '#');

p.gender = normalizeGender(p.gender);


// Contact

getline(fin, p.contact, '#');


// Balance
```

```
getline(fin, data);

p.balance = safeStod(data);

// Validation

if (p.patientId <= 0)

{

    continue;

}

if (p.name == "")

{

    continue;

}

if (!isValidPhone(p.contact))

{

    continue;

}

// Duplicate check

bool duplicate = false;

for (int i = 0; i < count; i++)

{

    if (temp[i].patientId == p.patientId)

    {

        duplicate = true;

        break;

    }

}
```

```

    }

    if (!duplicate)
    {
        temp[count] = p;
        count++;
    }
}

fin.close();

delete[] patients;

patients = new Patient[count];

for (int i = 0; i < count; i++)
{
    patients[i] = temp[i];
}

patientCount = count;
}

void savePatients()
{
    ofstream fout("patients.txt");

    for (int i = 0; i < patientCount; i++)
    {
        fout << patients[i].patientId << "#"

```

```
        << patients[i].name      << "#"
        << patients[i].age      << "#"
        << patients[i].gender   << "#"
        << patients[i].contact  << "#"
        << patients[i].balance  << endl;
    }
}
```

```
void loadDoctors()
```

```
{
    ifstream fin("doctors.txt");
    if (!fin)
    {
        return;
    }
}
```

```
    Doctor temp[500];
```

```
    int count = 0;
```

```
    while (!fin.eof())
```

```
    {
        Doctor d;
        string data;
```

```
// Doctor ID

getline(fin, data, '#');

if (data == "")
{
    break;
}

d.doc_id = stoi(data);

// Name

getline(fin, d.name, '#');

// Specialty

getline(fin, d.specialty, '#');

// Experience

getline(fin, data);

d.experience = safeStoi(data);

// Validation

if (d.doc_id <= 0)
{
    continue;
}
```

```
    }

    if (trim(d.name).empty())
    {
        continue;
    }

    temp[count] = d;
    count++;
}

fin.close();
delete[] doctors;
doctors = new Doctor[count];
for (int i = 0; i < count; i++)
{
    doctors[i] = temp[i];
}

doctorCount = count;
}

void saveDoctors()
{
    ofstream fout("doctors.txt");
```

```
    for (int i = 0; i < doctorCount; i++)

        fout << doctors[i].doc_id    << "#"

            << doctors[i].name        << "#"

            << doctors[i].specialty << "#"

            << doctors[i].experience <<endl;

}
```

```
void loadAppointments()

{

    ifstream fin("appointments.txt");

    if (!fin)

    {

        return;

    }

    Appointment temp[1000];

    int count = 0;

    while (!fin.eof())

    {

        Appointment a;

        string data;

        // Patient ID

        getline(fin, data, '#');
```

```
if (data == "")
{
    break;
}

a.patientId = safeStoi(data);

// Doctor ID
getline(fin, data, '#');

a.doctorId = safeStoi(data);

// Date
getline(fin, data, '#');

data = normalizeDateFormat(trim(data));

strncpy(a.date, data.c_str(), 10);
a.date[10] = '\\0';

// Time
getline(fin, data);

data = convertTo12Hour(trim(data));
```

```
    strncpy(a.time, data.c_str(), 9);
    a.time[9] = '\\0';

    // Validation
    if (a.patientId <= 0 || a.doctorId <= 0)
    {
        continue;
    }

    if (findPatientIndex(a.patientId) == -1)
    {
        continue;
    }

    if (findDoctorIndex(a.doctorId) == -1)
    {
        continue;
    }

    temp[count] = a;
    count++;
}

fin.close();
```

```

        delete[] appointments;

        appointments = new Appointment[count];

        for (int i = 0; i < count; i++)
        {
            appointments[i] = temp[i];
        }

        appointmentCount = count;
    }

void saveAppointments() {
    ofstream fout("appointments.txt");

    for (int i = 0; i < appointmentCount; i++)
        fout << appointments[i].patientId << "#"
              << appointments[i].doctorId  << "#"
              << appointments[i].date      << "#"
              << appointments[i].time      << "\n";
}

void loadTreatments()
{
    ifstream fin("treatments.txt");

    if (!fin)
    {
        return;
    }
}

```

```
Treatment temp[1000];

int count = 0;

while (!fin.eof())
{
    Treatment t;
    string data;

    // Patient ID
    getline(fin, data, '#');

    if (data == "")
    {
        break;
    }

    t.patientId = safeStoi(data);

    // Description
    getline(fin, t.description, '#');

    // Cost
    getline(fin, data, '#');
```

```
t.cost = safeStod(data);

// Paid status
getline(fin, data);

data = trim(data);

if (data == "true" || data == "1" || data == "Paid")
{
    t.paid = true;
}
else
{
    t.paid = false;
}

// Validation
if (t.patientId <= 0)
{
    continue;
}

if (findPatientIndex(t.patientId) == -1)
{
```

```
        continue;

    }

    temp[count] = t;

    count++;

}

fin.close();


delete[] treatments;

treatments = new Treatment[count];

for (int i = 0; i < count; i++)

{

    treatments[i] = temp[i];

}

treatmentCount = count;

}

void saveTreatments()

{

    ofstream fout("treatments.txt");

    for (int i = 0; i < treatmentCount; i++)

        fout << treatments[i].patientId << "#"

            << treatments[i].description << "#"

            << treatments[i].cost << "#"

            << (treatments[i].paid ? "true" : "false") << endl;

}
```

```
// _____  
// FILE CLEANING (runs at startup)  
// _____  
  
void cleanFile()  
{  
    cout << "\n[System] Running file cleaning & validation...\n";  
  
    // Load each file, apply all cleaning rules, then write back ONLY if  
    // records were actually loaded. This prevents overwriting a valid file  
    // with empty data if the file path is wrong or the file is unreadable.  
  
    loadPatients();  
    if (patientCount > 0)  
        savePatients();  
    else  
        cout << " [Warning] patients.txt: no valid records found or file  
missing. Original file NOT overwritten.\n";  
  
    loadDoctors();  
    if (doctorCount > 0)  
        saveDoctors();  
    else  
        cout << " [Warning] doctors.txt: no valid records found or file  
missing. Original file NOT overwritten.\n";  
}
```

```

loadAppointments();

if (appointmentCount > 0)
    saveAppointments();
else
    cout << " [Warning] appointments.txt: no valid records found or
file missing. Original file NOT overwritten.\n";

loadTreatments();

if (treatmentCount > 0)
    saveTreatments();
else
    cout << " [Warning] treatments.txt: no valid records found or file
missing. Original file NOT overwritten.\n";

cout << "[System] File cleaning complete.\n";

cout << "  Loaded: " << patientCount    << " patients,  "
               << doctorCount    << " doctors,  "
               << appointmentCount << " appointments,  "
               << treatmentCount  << " treatments.\n";
}

// _____
//  LOOKUP HELPERS
// _____

```

```
int findPatientIndex(int id)
{
    for (int i = 0; i < patientCount; i++)
        if (patients[i].patientId == id)
            return i;
    return -1;
}
```

```
int findDoctorIndex(int id)
{
    for (int i = 0; i < doctorCount; i++)
        if (doctors[i].doc_id == id)
            return i;
    return -1;
}
```

// Print a single patient row

```
void printPatient(const Patient& p)
{
    cout << left
          << setw(6) << p.patientId
          << setw(22) << p.name
          << setw(5) << p.age
          << setw(9) << p.gender
```

```

        << setw(14) << p.contact

        << "PKR " << fixed << setprecision(0) << p.balance << "\n";
    }

void printPatientHeader()
{
    cout << string(70, '-') << endl;
    cout << left

        << setw(6) << "ID"
        << setw(22) << "Name"
        << setw(5) << "Age"
        << setw(9) << "Gender"
        << setw(14) << "Contact"
        << "Balance"<< endl;

    cout << string(70, '-') << endl;
}

void printDoctor(const Doctor& d)
{
    cout << left

        << setw(6) << d.doc_id
        << setw(22) << d.name
        << setw(22) << d.specialty
        << d.experience << " yrs"<< endl;
}

```

```
void printDoctorHeader()
{
    cout << string(60, '-') << endl;
    cout << left
        << setw(6) << "ID"
        << setw(22) << "Name"
        << setw(22) << "Specialty"
        << "Exp"<< endl;
    cout << string(60, '-') << endl;
}

// -----
// PATIENT MANAGEMENT
// -----

void viewPatients()
{
    if (patientCount == 0)
    {
        cout << "No patient records found."<< endl;
        return;
    }

    printPatientHeader();
    for (int i = 0; i < patientCount; i++)
        printPatient(patients[i]);
}
```

```
        cout << "Total: " << patientCount << " patient(s)." << endl;
    }

void addPatient()
{
    Patient p;

    cout << "\n--- Add New Patient ---\n";

    cout << "Enter Patient ID: ";

    cin >> p.patientId;

    if (p.patientId <= 0)
    {
        cout << "Invalid ID." << endl;

        return;
    }

    if (findPatientIndex(p.patientId) != -1)
    {
        cout << "Patient ID already exists." << endl;

        return;
    }

    cin.ignore();

    cout << "Name: ";

    getline(cin, p.name);

    p.name = trim(p.name);

    if (p.name.empty())
```

```
{
    cout << "Name cannot be empty."<<endl;
    return;
}

cout << "Age: ";

    cin >> p.age;
cin.ignore();

cout << "Gender (Male/Female): ";

    getline(cin, p.gender);
    p.gender = normalizeGender(p.gender);

cout << "Contact (11 digits): ";

    getline(cin, p.contact);
    p.contact = trim(p.contact);
if (!isValidPhone(p.contact))
{
    cout << "Invalid phone number. Patient not added."<<endl;
    return;
}

cout << "Balance (PKR): ";

    cin >> p.balance;
    cin.ignore();

// Resize array

Patient* newArr = new Patient[patientCount + 1];
```

```
    for (int i = 0; i < patientCount; i++)
        newArr[i] = patients[i];
    newArr[patientCount] = p;
    delete[] patients;
    patients = newArr;
    patientCount++;
    savePatients();
    cout << "Patient added successfully."<<endl;
}

void updatePatient()
{
    int id;
    cout << "\nEnter Patient ID to update: ";
    cin >> id;
    cin.ignore();
    int idx = findPatientIndex(id);
    if (idx == -1)
    {
        cout << "Patient not found."<<endl;
        return;
    }
    Patient& p = patients[idx];
    string inp;
```

```

    cout << "Current Name: " << p.name << "    New Name (Enter to keep): ";

    getline(cin, inp);

    if (!trim(inp).empty()) p.name = trim(inp);

    int a;

    cout << "Current Age: " << p.age << "    New Age (0 to keep): ";

    cin >> a;

    cin.ignore();

    if (a > 0)

        p.age = a;

    cout << "Current Gender: " << p.gender << "    New Gender (Enter to
keep): ";

    getline(cin, inp);

    if (!trim(inp).empty()) p.gender = normalizeGender(inp);

    cout << "Current Contact: " << p.contact << "    New Contact (Enter to
keep): ";

    getline(cin, inp); inp = trim(inp);

    if (!inp.empty())

    {

        if (!isValidPhone(inp))

        {

            cout << "Invalid phone. Contact not updated."<<endl;

        }

        else p.contact = inp;

    }

    double b;

```

```
    cout << "Current Balance: " << p.balance << "    New Balance: ";  
  
    cin >> b;  
  
    cin.ignore();  
  
    if (b >= 0)  
        p.balance = b;  
  
    savePatients();  
  
    cout << "Patient updated successfully.\n";  
}
```

```
void deletePatient()  
{  
    int id;  
  
    cout << "\nEnter Patient ID to delete: ";  
  
    cin >> id;  
  
    cin.ignore();  
  
    int idx = findPatientIndex(id);  
  
    if (idx == -1)  
    {  
        cout << "Patient not found."<<endl;  
  
        return;  
    }  
  
    Patient* newArr = new Patient[patientCount - 1];  
  
    int j = 0;  
  
    for (int i = 0; i < patientCount; i++)
```

```

        if (i != idx)

            newArr[j++] = patients[i];

        delete[] patients;

        patients = newArr;

        patientCount--;

        savePatients();

        cout << "Patient deleted.\n";
    }

// -----
// DOCTOR MANAGEMENT
// -----

void viewDoctors()
{
    if (doctorCount == 0)
    {
        cout << "No doctor records found."<<endl;

        return;

    }

    printDoctorHeader();

    for (int i = 0; i < doctorCount; i++)

        printDoctor(doctors[i]);

    cout << "Total: " << doctorCount << " doctor(s)."<<endl;
}

```

```
void addDoctor()
{
    Doctor d;

    cout << "\n--- Add New Doctor ---\n";

    cout << "Doctor ID: ";

    cin >> d.doc_id;

    cin.ignore();

    if (d.doc_id <= 0)
    {
        cout << "Invalid ID."<<endl;

        return;
    }

    if (findDoctorIndex(d.doc_id) != -1)
    {
        cout << "Doctor ID already exists."<<endl;

        return;
    }

    cout << "Name: ";

    getline(cin, d.name);

    d.name = trim(d.name);

    if (d.name.empty())
    {
        cout << "Name cannot be empty."<<endl;
    }
}
```

```
        return;

    }

    cout << "Specialty: ";

    getline(cin, d.specialty);

    cout << "Years of Experience: ";

    cin >> d.experience;

    cin.ignore();


    Doctor* newArr = new Doctor[doctorCount + 1];

    for (int i = 0; i < doctorCount; i++) newArr[i] = doctors[i];

    newArr[doctorCount] = d;

    delete[] doctors;

    doctors = newArr;

    doctorCount++;

    saveDoctors();

    cout << "Doctor added successfully."<<endl;
}

void updateDoctor()
{
    int id;

    cout << "\nEnter Doctor ID to update: ";

    cin >> id;

    cin.ignore();
```

```
int idx = findDoctorIndex(id);

if (idx == -1)

{

    cout << "Doctor not found."<<endl;

    return;

}

Doctor& d = doctors[idx];

cout << "Current Name: " << d.name << " New Name (Enter to keep): ";

string inp;

    getline(cin, inp);

if (!trim(inp).empty())

    d.name = trim(inp);

    cout << "Current Specialty: " << d.specialty << " New Specialty (Enter to keep): ";

    getline(cin, inp);

if (!trim(inp).empty())

    d.specialty = trim(inp);

    cout << "Current Experience: " << d.experience << " yrs New Experience (0 to keep): ";

    int e; cin >> e; cin.ignore();

if (e > 0)

    d.experience = e;

saveDoctors();

cout << "Doctor updated successfully."<<endl;

}
```

```
void deleteDoctor()
{
    int id;

    cout << "\nEnter Doctor ID to delete: ";

    cin >> id;

    cin.ignore();

    int idx = findDoctorIndex(id);

    if (idx == -1)
    {
        cout << "Doctor not found."<<endl;

        return;
    }

    Doctor* newArr = new Doctor[doctorCount - 1];

    int j = 0;

    for (int i = 0; i < doctorCount; i++) if (i != idx) newArr[j++] =
doctors[i];

    delete[] doctors;

    doctors = newArr;

    doctorCount--;

    saveDoctors();

    cout << "Doctor deleted.\n";
}
```

```
// _____  
//  APPOINTMENTS  
//  _____  
  
void viewAppointments()  
{  
    if (appointmentCount == 0)  
    {  
        cout << "No appointments found."<<endl;  
        return;  
    }  
  
    cout << string(65, '-') <<endl;  
  
    cout << left << setw(8) << "Pat ID" << setw(8) << "Doc ID"  
        << setw(14) << "Date" << setw(12) << "Time" <<endl;  
  
    cout << string(65, '-') <<endl;  
  
    for (int i = 0; i < appointmentCount; i++)  
    {  
        cout << left << setw(8) << appointments[i].patientId  
            << setw(8) << appointments[i].doctorId  
            << setw(14) << appointments[i].date  
            << setw(12) << appointments[i].time <<endl;  
    }  
  
    cout << "Total: " << appointmentCount << " appointment(s)."<<endl;  
}
```

```

// Predefined slots for the weekly schedule
const char* WEEKLY_SLOTS[] =
{
    "09:00 AM", "10:00 AM", "11:00 AM", "12:00 PM",
    "01:00 PM", "02:00 PM", "03:00 PM", "04:00 PM"
};

const int SLOT_COUNT = 8;

// Days in a simple week (offset from base date – for bonus display)
const char* WEEK_DAYS[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};

bool isSlotTaken(int docId, const char* date, const char* time)
{
    for (int i = 0; i < appointmentCount; i++)
    {
        if (appointments[i].doctorId == docId &&
            strcmp(appointments[i].date, date) == 0 &&
            strcmp(appointments[i].time, time) == 0)
        {
            return true;
        }
    }
}

```

```
        return false;
    }

    int findNextFreeSlot(int docId, const char* date, int startSlot)
    {
        for (int i = startSlot; i < SLOT_COUNT; i++)
        {
            // If slot is free
            if (!isSlotTaken(docId, date, WEEKLY_SLOTS[i]))
            {
                return i;
            }
        }

        // No free slot found
        return -1;
    }

    void scheduleAppointment()
    {
        int patId, docId;

        cout << "\n--- Schedule Appointment ---\n";

        cout << "Patient ID: ";
```

```
    cin >> patId;

    cin.ignore();

    int pIdx = findPatientIndex(patId);
    if (pIdx == -1)
    {
        cout << "Patient not found."<<endl;
        return;
    }

    cout << "Doctor ID: ";

    cin >> docId;

    cin.ignore();

    int dIdx = findDoctorIndex(docId);
    if (dIdx == -1)
    {
        cout << "Doctor not found."<<endl;
        return;
    }

    string dateStr, timeStr;

    cout << "Date (MM-DD-YYYY): ";

    getline(cin, dateStr);

    dateStr = normalizeDateFormat(trim(dateStr));

    cout << "Time (e.g. 10:00 AM): ";
```

```

getline(cin, timeStr);

timeStr = convertTo12Hour(trim(timeStr));

// Check balance against cheapest related treatment cost (use 0 if none
yet)

// For simplicity: check slot availability

char date[11], time[10];

strncpy(date, dateStr.c_str(), 10);

date[10] = '\0';

strncpy(time, timeStr.c_str(), 9);

time[9] = '\0';

if (isSlotTaken(docId, date, time))

{

    cout << "Slot already booked! Searching for next available slot
(same day)...\n";

    // Find slot index of requested time

    int reqSlot = -1;

    for (int i = 0; i < SLOT_COUNT; i++)

        if (strcmp(WEEKLY_SLOTS[i], time) == 0)

            {

                reqSlot = i;

                break;

            }

}

```

```

        int nextSlot = findNextFreeSlot(docId, date, (reqSlot >= 0 ?
reqSlot + 1 : 0));

        if (nextSlot == -1)
        {
            cout << "No available slot for this doctor on this day."<<endl;

            return;
        }

        cout << "Next available slot: " << WEEKLY_SLOTS[nextSlot] <<endl;
        cout << "Book this slot? (y/n): ";

        char ch;

        cin >> ch;

        cin.ignore();

        if (ch != 'y' && ch != 'Y')
        {
            cout << "Appointment not scheduled."<<endl;

            return;
        }

        strncpy(time, WEEKLY_SLOTS[nextSlot], 9);

        time[9] = '\0';
    }

    Appointment a;

    a.patientId = patId;

    a.doctorId = docId;

```

```

    strncpy(a.date, date, 10);

    a.date[10] = '\0';

    strncpy(a.time, time, 9);

    a.time[9] = '\0';


Appointment* newArr = new Appointment[appointmentCount + 1];
for (int i = 0; i < appointmentCount; i++) newArr[i] = appointments[i];
newArr[appointmentCount] = a;

delete[] appointments;

appointments = newArr;

appointmentCount++;

saveAppointments();

cout << "Appointment scheduled: Patient " << patId
      << " with Doctor " << docId << " on " << date << " at " << time <<
"\n";
}

void cancelAppointment()
{
    int patId, docId;

    cout << "\nCancel Appointment\n";

    cout << "Patient ID: ";

    cin >> patId;

    cout << "Doctor ID: ";

```

```
    cin >> docId;

    cin.ignore();

    string dateStr;

    cout << "Date (MM-DD-YYYY): ";

    getline(cin, dateStr);

    dateStr = normalizeDateFormat(trim(dateStr));


    int found = -1;

    for (int i = 0; i < appointmentCount; i++)

        if (appointments[i].patientId == patId &&

            appointments[i].doctorId == docId &&

            string(appointments[i].date) == dateStr)

            {

                found = i;

                break;

            }

    if (found == -1)

    {

        cout << "Appointment not found."<<endl;

        return;

    }


    Appointment* newArr = new Appointment[appointmentCount - 1];
```

```
int j = 0;

for (int i = 0; i < appointmentCount; i++) if (i != found) newArr[j++]
= appointments[i];

delete[] appointments;

appointments = newArr;

appointmentCount--;

saveAppointments();

cout << "Appointment cancelled."<<endl;
}

// Weekly schedule table
void viewWeeklySchedule()
{
    int docId;

    cout << "\nEnter Doctor ID for weekly schedule: ";

    cin >> docId;

    cin.ignore();

    if (findDoctorIndex(docId) == -1)
    { cout << "Doctor not found.\n";

      return;

    }

    cout << "\n--- Weekly Schedule for Doctor " << docId << " ---\n";

    cout << left << setw(12) << "Time";
```

```

    cout << "Appointments"<<endl;

    cout << string(60, '-') <<endl;

    for (int s = 0; s < SLOT_COUNT; s++)
    {
        cout << setw(12) << WEEKLY_SLOTS[s];

        bool any = false;

        for (int i = 0; i < appointmentCount; i++)
        {
            if (appointments[i].doctorId == docId &&
                strcmp(appointments[i].time, WEEKLY_SLOTS[s]) == 0)
            {
                cout << "Pat#" << appointments[i].patientId << " on " <<
appointments[i].date << " ";

                any = true;
            }
        }

        if (!any)
            cout << "Free";

        cout<<endl;

    }
}

// _____

```

```
// TREATMENTS & BILLING

// -----

void viewTreatments()
{
    if (treatmentCount == 0)
    {
        cout << "No treatment records.\n";
        return;
    }

    int id;

    cout << "Enter Patient ID (0 for all): ";

    cin >> id;

    cin.ignore();

    bool found = false;

    cout << string(70, '-') << endl;

    cout << left << setw(8) << "Pat ID" << setw(30) << "Description"
        << setw(12) << "Cost (PKR)" << "Status"<< endl;

    cout << string(70, '-') << endl;

    for (int i = 0; i < treatmentCount; i++)
    {
        if (id == 0 || treatments[i].patientId == id)
        {
            cout << left << setw(8) << treatments[i].patientId
                << setw(30) << treatments[i].description
```

```

        << setw(12) << fixed << setprecision(0) <<
treatments[i].cost

        << (treatments[i].paid ? "Paid" : "Unpaid") <<endl;

        found = true;

    }

}

if (!found)

    cout << "No records found."<<endl;

}

void addTreatment()

{

    Treatment t;

    cout << "\n--- Add Treatment ---\n";

    cout << "Patient ID: ";

        cin >> t.patientId;

        cin.ignore();

    if (findPatientIndex(t.patientId) == -1)

        {

            cout << "Patient not found."<<endl;

            return;

        }

    cout << "Treatment Description: ";

        getline(cin, t.description);

```

```
cout << "Cost (PKR): ";

    cin >> t.cost;

    cin.ignore();

cout << "Is bill paid? (y/n): ";

    char ch;

    cin >> ch;

    cin.ignore();

t.paid = (ch == 'y' || ch == 'Y');

// Check patient balance if paying now
if (t.paid)
{
    int pIdx = findPatientIndex(t.patientId);

    if (patients[pIdx].balance < t.cost)
    {
        cout << "Insufficient balance (Balance: PKR " <<
patients[pIdx].balance
        << ", Cost: PKR " << t.cost << "). Cannot mark as
paid."<<endl;

        t.paid = false;
    }

    else
    {
        patients[pIdx].balance -= t.cost;

        savePatients();
    }
}
```

```

        cout << "Amount deducted. New balance: PKR " <<
patients[pIdx].balance <<endl;

    }

}

Treatment* newArr = new Treatment[treatmentCount + 1];
for (int i = 0; i < treatmentCount; i++) newArr[i] = treatments[i];
newArr[treatmentCount] = t;
delete[] treatments;
treatments = newArr;
treatmentCount++;
saveTreatments();
cout << "Treatment record added.\n";
}

void updatePayment()
{
    int patId;
    cout << "\nUpdate Payment Status\n";
    cout << "Patient ID: ";
    cin >> patId;
    cin.ignore();
    string desc;
    cout << "Treatment description: ";

```

```

        getline(cin, desc);

    bool found = false;

    for (int i = 0; i < treatmentCount; i++)
    {
        if (treatments[i].patientId == patId &&
            treatments[i].description.find(desc) != string::npos)
        {
            if (treatments[i].paid)
            {
                cout << "Already marked Paid."<<endl;
                continue;
            }

            int pIdx = findPatientIndex(patId);

            if (pIdx != -1 && patients[pIdx].balance < treatments[i].cost)
            {
                cout << "Insufficient balance (PKR " <<
patients[pIdx].balance
                << "). Cannot pay PKR " << treatments[i].cost <<endl;
                continue;
            }

            treatments[i].paid = true;

            if (pIdx != -1)
            {
                patients[pIdx].balance -= treatments[i].cost;

                savePatients();
            }
        }
    }
}

```

```

        }

        found = true;

        cout << "Payment updateId for: " << treatments[i].description
<<endl;

    }

}

if (!found) cout << "No matching unpaid treatment found."<<endl;

saveTreatments();
}

void generateBill()
{
    int patId;

    cout << "\nGenerate Bill for Patient ID: ";

    cin >> patId;

    cin.ignore();

    int pIdx=findPatientIndex(patId);

    if (pIdx == -1)
    {
        cout << "Patient not found."<<endl;

        return;

    }

    cout << "\n=====\\n";

```

```

cout << "          HOSPITAL BILL SUMMARY\n";

cout << "=====\n";

cout << "Patient: " << patients[pIdx].name
        << " (ID: " << patId << ")\n";

cout << "Contact: " << patients[pIdx].contact << "\n";

cout << string(60, '-') << "\n";

cout << left << setw(28) << "Treatment" << setw(12) << "Cost" <<
"Status\n";

cout << string(60, '-') << "\n";


double totalCost = 0, paidAmt = 0;

bool anyRecord = false;

for (int i = 0; i < treatmentCount; i++)
{
    if (treatments[i].patientId == patId)
    {
        cout << left << setw(28) << treatments[i].description
                << setw(12) << fixed << setprecision(0) <<
treatments[i].cost
                << (treatments[i].paid ? "Paid" : "Unpaid") << endl;

        totalCost += treatments[i].cost;

        if (treatments[i].paid) paidAmt += treatments[i].cost;

        anyRecord = true;
    }
}
}

```

```

    if (!anyRecord)

        { cout << "No treatments on record."<<endl;

          return;

        }

    cout << string(60, '-') <<endl;

    cout << right << setw(28) << "Total Cost: " << "PKR " << totalCost
    <<endl;

    cout << right << setw(28) << "Amount Paid: " << "PKR " << paidAmt
    <<endl;

    cout << right << setw(28) << "Outstanding: " << "PKR " << (totalCost -
    paidAmt) <<endl;

    cout << "=====\n";
}

// -----
//  SEARCH / REPORTS
// -----

void searchPatientsBydoc_ID()
{

    int id;

    string name;

    cout << "\nSearch Patient by:\n 1. ID\n 2. Name\nChoice: ";

    int ch;

    cin >> ch;

    cin.ignore();

```

```
if (ch == 1)
{
    cout << "Enter Patient ID: ";

    cin >> id;

    cin.ignore();

    int idx = findPatientIndex(id);

    if (idx == -1)
    { cout << "Patient not found.\n";
      return;
    }

    printPatientHeader();

    printPatient(patients[idx]);
}

else
{
    cout << "Enter name: ";

    getline(cin, name);

    printPatientHeader();

    bool found = false;

    for (int i = 0; i < patientCount; i++)
    {
        if (patients[i].name.find(name) != string::npos)
        {
            printPatient(patients[i]);
        }
    }
}
```

```
        found = true;

    }

}

if (!found)

    cout << "No patients found."<<endl;

}

}

void searchDoctorBySpecialty()

{

    int ch;

    cout << "\nSearch Doctor by:\n 1. ID\n 2. Specialty\nChoice: ";

    cin >> ch;

    cin.ignore();

    printDoctorHeader();

    if (ch == 1)

    {

        int id;

        cout << "Doctor ID: ";

        cin >> id;

        cin.ignore();

        int idx = findDoctorIndex(id);

        if (idx == -1)

            cout << "Doctor not found."<<endl;

        else printDoctor(doctors[idx]);

    }

}
```

```

    }

    else
    {
        string spec;

        cout << "Specialty : ";

        getline(cin, spec);

        bool found = false;

        for (int i = 0; i < doctorCount; i++)
        {
            if (doctors[i].specialty.find(spec) != string::npos)
            {
                printDoctor(doctors[i]); found = true;
            }
        }

        if (!found)

            cout << "No doctors found for that specialty."<<endl;

    }
}

```

```

void viewTreatmentsByDoctor()
{
    int docId;

    cout << "\nSearch treatments by Doctor ID: ";
}

```

```
cin >> docId;

cin.ignore();

int dIdx = findDoctorIndex(docId);

if (dIdx == -1)
{
    cout << "Doctor not found." << endl;
    return;
}

    cout << "-----"
-----" << endl;

    cout << "Pat ID\tDescription\t\tCost\tStatus" << endl;

    cout << "-----"
-----" << endl;

bool found = false;

// Check every treatment
for (int t = 0; t < treatmentCount; t++)
{
    int pid = treatments[t].patientId;
```

```
bool belongsToDoctor = false;

// Check if patient had appointment with this doctor
for (int a = 0; a < appointmentCount; a++)
{
    if (appointments[a].doctorId == docId &&
        appointments[a].patientId == pid)
    {
        belongsToDoctor = true;
        break;
    }
}

// Print treatment only once
if (belongsToDoctor)
{
    cout << treatments[t].patientId << "\t"
         << treatments[t].description << "\t\t"
         << treatments[t].cost << "\t";

    if (treatments[t].paid)
    {
        cout << "Paid" << endl;
    }
}
```

```

        else
        {
            cout << "Unpaid" << endl;
        }

        found = true;
    }
}

if (!found)
{
    cout << "No treatments found for this doctor." << endl;
}
}

// Sort doctors by experience (bubble sort)
void sortDoctorsByExperience()
{
    for (int i = 0; i < doctorCount - 1; i++)
        for (int j = 0; j < doctorCount - i - 1; j++)
            if (doctors[j].experience > doctors[j + 1].experience)
            {
                Doctor tmp = doctors[j];
                doctors[j] = doctors[j + 1];
                doctors[j + 1] = tmp;
            }
}

```

```

    }

    cout << "\nDoctors sorted by experience (ascending):\n";

    printDoctorHeader();

    for (int i=0;i<doctorCount;i++)
        printDoctor(doctors[i]);

    saveDoctors();
}

void removeDuplicatePatients()
{
    loadPatients();

    cout<<"Duplicate check complete. Active records: " << patientCount
<<endl;
}

// -----
//  MENUS
// -----

void patientMenu()
{
    int ch;

    do {

        cout << "\n--- Patient Management ---\n";

        cout << "1. Add Patient\n2. Update Patient\n3. Delete Patient\n4.
View All Patients\n0. Back\n";

        cout << "Choice: "; cin >> ch; cin.ignore();

```

```

        switch (ch) {

            case 1: addPatient(); break;

            case 2: updatePatient(); break;

            case 3: deletePatient(); break;

            case 4: viewPatients(); break;

            case 0: break;

            default: cout << "Invalid choice.\n";

        }

    } while (ch != 0);

}

void doctorMenu()
{

    int ch;

    do {

        cout << "\n--- Doctor Management ---\n";

        cout << "1. Add Doctor\n2. Update Doctor\n3. Delete Doctor\n4. View
All Doctors\n0. Back\n";

        cout << "Choice: ";

        cin >> ch;

        cin.ignore();

        switch (ch)

        {

            case 1: addDoctor();

```

```

        break;

        case 2: updateDoctor();

        break;

        case 3: deleteDoctor();

        break;

        case 4: viewDoctors();

        break;

        case 0: break;

        default: cout << "Invalid choice.\n";

    }

} while (ch != 0);

}

void appointmentMenu() {

    int ch;

    do {

        cout << "\n--- Appointments ---\n";

        cout << "1. Schedule Appointment\n2. Cancel Appointment\n3. View  
All Appointments\n";

        cout << "4. Weekly Schedule \n0. Back\n";

        cout << "Choice: ";

        cin >> ch;

        cin.ignore();

        switch (ch)

```

```

        {
            case 1: scheduleAppointment(); break;
            case 2: cancelAppointment(); break;
            case 3: viewAppointments(); break;
            case 4: viewWeeklySchedule(); break;
            case 0: break;
            default: cout << "Invalid choice.\n";
        }
    } while (ch != 0);
}

void treatmentMenu()
{
    int ch;
    do {
        cout << "\n--- Treatments & Billing ---\n";
        cout << "1. Add Treatment Record\n2. View Treatments\n";
        cout << "3. Update Payment Status\n4. Generate Bill\n0. Back\n";
        cout << "Choice: "; cin >> ch; cin.ignore();
        switch (ch) {
            case 1: addTreatment();
                    break;
            case 2: viewTreatments();
                    break;

```

```
        case 3: updatePayment();

            break;

        case 4: generateBill();

            break;

        case 0: break;

        default: cout << "Invalid choice.\n";

    }

} while (ch != 0);

}

void searchMenu()

{

    int ch;

    do {

        cout << "\n--- Search / Reports ---\n";

        cout << "1. Search Patient (by ID or Name)\n";

        cout << "2. Search Doctor (by ID or Specialty)\n";

        cout << "3. View Treatments by Doctor\n";

        cout << "4. Sort Doctors by Experience\n";

        cout << "5. Remove Duplicate Patients (re-scan)\n";

        cout << "0. Back\n";

        cout << "Choice: ";

        cin >> ch;

        cin.ignore();

    }
```

```

        switch (ch)
        {
            case 1: searchPatientsBydoc_ID();

                    break;

            case 2: searchDoctorBySpecialty();

                    break;

            case 3: viewTreatmentsByDoctor();

                    break;

            case 4: sortDoctorsByExperience();

                    break;

            case 5: removeDuplicatePatients();

                    break;

            case 0: break;

            default: cout << "Invalid choice.\n";

        }

    } while (ch != 0);
}

// -----
//  LOGIN
// -----

bool login()
{
    string empId, pass;

```

```

    cout << "\n=====\\n";

    cout << "          HOSPITAL MANAGEMENT SYSTEM\\n";

    cout << "=====\\n";

    cout << "Please login as Employee\\n";

    cout << "Enter Employee ID: ";

        getline(cin, empId);

    cout << "Enter Password: ";

        getline(cin, pass);

    if (trim(empId) == EMP_ID && trim(pass) == EMP_PASS)

        {

            cout << "Login Successful!\\n";

            return true;

        }

    cout << "Invalid credentials. Access denied.\\n";

    return false;

}

// -----
//  MAIN
// -----

int main()

{

    cleanFile(); // Always run file cleaning first

```

```
if (!login())  
    return 0;  
  
int choice;  
do  
{  
    cout << "\n===== Main Menu =====\n";  
    cout << "1. Patient Management\n";  
    cout << "2. Doctor Management\n";  
    cout << "3. Appointments\n";  
    cout << "4. Treatments & Billing\n";  
    cout << "5. Search / Reports\n";  
    cout << "6. Exit\n";  
    cout << "=====\n";  
    cout << "Enter your choice: ";  
    cin >> choice;  
    cin.ignore();  
    switch (choice) {  
        case 1: patientMenu();  
            break;  
        case 2: doctorMenu();  
            break;  
        case 3: appointmentMenu();  
            break;
```

```
        case 4: treatmentMenu();

            break;

        case 5: searchMenu();

            break;

        case 6: cout << "Exiting. Goodbye!\n";

            break;

        default: cout << "Invalid choice. Please try again.\n";

    }

} while (choice != 6);


// Free dynamic memory

delete[] patients;

delete[] doctors;

delete[] appointments;

delete[] treatments;

system("pause");

return 0;

}
```

OUTPUT SCREENSHOTS

```
[System] Running file cleaning & validation...
[System] File cleaning complete.
    Loaded: 134 patients, 35 doctors, 55 appointments, 58 treatments.

=====
          HOSPITAL MANAGEMENT SYSTEM
=====
Please login as Employee
Enter Employee ID: 123
Enter Password: pass123
Login Successful!

===== Main Menu =====
1. Patient Management
2. Doctor Management
3. Appointments
4. Treatments & Billing
5. Search / Reports
6. Exit
=====
Enter your choice: 1

--- Patient Management ---
1. Add Patient
2. Update Patient
3. Delete Patient
4. View All Patients
0. Back
Choice: 1

--- Add New Patient ---
Enter Patient ID: 400
Name: Bisma Khan
Age: 18
```

Gender (Male/Female): F
Contact (11 digits): 03326850000
Balance (PKR): 8900000
Patient added successfully.

--- Patient Management ---

1. Add Patient
2. Update Patient
3. Delete Patient
4. View All Patients
0. Back

Choice: 2

Enter Patient ID to update: 102

Current Name: Sara Ahmed New Name (Enter to keep): Hamza ali

Current Age: 30 New Age (0 to keep): 45

Current Gender: Female New Gender (Enter to keep): M

Current Contact: 03009876543 New Contact (Enter to keep): 03335044256

Current Balance: 12000 New Balance: 230000

Patient updated successfully.

--- Patient Management ---

1. Add Patient
2. Update Patient
3. Delete Patient
4. View All Patients
0. Back

Choice: 3

Enter Patient ID to delete: 400

Patient deleted.

--- Patient Management ---

1. Add Patient
2. Update Patient

3. Delete Patient
4. View All Patients
0. Back
Choice: 4

ID	Name	Age	Gender	Contact	Balance
101	Ali Raza	25	Male	03001234567	PKR 5000
102	Hamza ali	45	Male	03335044256	PKR 230000
103	Usman Khan	40	Male	03211234567	PKR 8000
104	Fatima Malik	22	Female	03451234567	PKR 3000
105	Hassan Ali	28	Male	03331234567	PKR 15000
106	Ayesha Noor	35	Female	03001239876	PKR 7000
107	Bilal Ahmed	45	Male	03211239876	PKR 20000
108	Zara Hussain	27	Female	03009871234	PKR 5500
109	Kamran Sheikh	50	Male	03451239876	PKR 10000
110	Nadia Iqbal	33	Female	03331239876	PKR 6000
111	Tariq Mehmood	38	Male	03001238765	PKR 4500
114	Hina Qureshi	29	Female	03451238765	PKR 2500
115	Asad Mirza	31	Male	03331238765	PKR 11000
116	Maryam Shah	26	Female	03001237654	PKR 7500
117	Faisal Javed	47	Male	03211237654	PKR 18000
118	Rabia Anwar	36	Female	03009877654	PKR 4000
119	Omer Farooq	23	Male	03451237654	PKR 6500
120	Sana Tariq	32	Female	03331237654	PKR 14000
121	Waseem Akram	44	Male	03001236543	PKR 9500
122	Lubna Butt	28	Female	03211236543	PKR 3500
123	Naveed Chaudhry	37	Male	03009876543	PKR 16000
124	Amna Siddiqui	21	Female	03451236543	PKR 2000
125	Shehzad Ali	55	Male	03331236543	PKR 22000
126	Kiran Bano	34	Female	03001235432	PKR 8500
127	Zahid Hussain	41	Male	03211235432	PKR 12500
128	Samina Malik	26	Female	03009875432	PKR 5000
129	Adil Khan	39	Male	03451235432	PKR 17000
130	Bushra Naz	30	Female	03331235432	PKR 6000

131	Rizwan Akhtar	48	Male	03001234321	PKR 19000
132	Mehwish Hayat	25	Female	03211234321	PKR 4000
133	Shoaib Akhtar	43	Male	03009874321	PKR 10500
134	Anum Shahid	27	Female	03451234321	PKR 3000
135	Junaid Malik	52	Male	03331234321	PKR 25000
136	Naila Inayat	31	Female	03001233210	PKR 7000
137	Waqas Gill	36	Male	03211233210	PKR 11000
138	Saira Batool	23	Female	03009873210	PKR 4500
139	Khurram Shahzad	46	Male	03451233210	PKR 15500
140	Fouzia Ashraf	38	Female	03331233210	PKR 8000
141	Asim Raza	29	Male	03001232109	PKR 6500
142	Tahira Bibi	44	Female	03211232109	PKR 9000
143	Mudassar Nazar	33	Male	03009872109	PKR 13500
144	Uzma Tauseef	27	Female	03451232109	PKR 3500
145	Sajid Mehmood	57	Male	03331232109	PKR 28000
146	Madiha Shah	22	Female	03001231098	PKR 2500
147	Zulfiqar Ali	49	Male	03211231098	PKR 21000
148	Parisa Khan	35	Female	03009871098	PKR 7500
149	Babar Azam	26	Male	03451231098	PKR 5500
150	Noor Fatima	30	Female	03331231098	PKR 10000
151	Hamid Raza	53	Male	03001230987	PKR 23000
152	Tanzila Bibi	31	Female	03211230987	PKR 6000
153	Shahzaib Khan	19	Male	03009870987	PKR 3000
154	Rukhsana Begum	60	Female	03451230987	PKR 9500
155	Danish Mehmood	34	Male	03331230987	PKR 12000
156	Sadia Kanwal	28	Female	03001229876	PKR 4500
157	Nawaz Sharif	62	Male	03211229876	PKR 30000
158	Zainab Fatima	20	Female	03009869876	PKR 2000
159	Pervaiz Ahmed	45	Male	03451229876	PKR 17500
160	Munazza Parveen	39	Female	03331229876	PKR 8500
161	Jawad Iqbal	33	Male	03001228765	PKR 7000
162	Shaista Malik	24	Female	03211228765	PKR 3500
163	Irfan Ali	41	Male	03009868765	PKR 11000
164	Samra Batool	36	Female	03451228765	PKR 6500
165	Ghulam Mustafa	58	Male	03331228765	PKR 26000

168	Farah Deebea	29	Female	03009867654	PKR 5000
169	Rehan Siddiqui	38	Male	03451227654	PKR 13000
170	Qurat ul Ain	25	Female	03331227654	PKR 4000
171	Tauseef Ahmed	51	Male	03001226543	PKR 24000
172	Huma Shabbir	32	Female	03211226543	PKR 7500
173	Arif Hussain	43	Male	03009866543	PKR 16000
174	Rubia Kanwal	27	Female	03451226543	PKR 3500
175	Zeeshan Malik	35	Male	03331226543	PKR 10500
176	Amina Bibi	55	Female	03001225432	PKR 20000
177	Kashif Nawaz	30	Male	03211225432	PKR 5500
178	Shazia Parveen	42	Female	03009865432	PKR 12000
179	Muneeb ur Rehman	24	Male	03451225432	PKR 2500
180	Gulnaz Akhtar	48	Female	03331225432	PKR 18000
181	Saeed Ahmed	37	Male	03001224321	PKR 9000
182	Nasrin Bibi	33	Female	03211224321	PKR 6000
183	Waheed Anwar	46	Male	03009864321	PKR 15000
184	Mariam Ashraf	21	Female	03451224321	PKR 2000
185	Azhar Mehmood	54	Male	03331224321	PKR 27000
186	Sumbul Fatima	28	Female	03001223210	PKR 5000
187	Nadeem Akhtar	40	Male	03211223210	PKR 11500
188	Raheela Bano	35	Female	03009863210	PKR 7000
189	Wajid Ali	29	Male	03451223210	PKR 4000
190	Farida Khanum	63	Female	03331223210	PKR 32000
191	Salman Khalid	22	Male	03001222109	PKR 3000
192	Tabassum Naz	38	Female	03211222109	PKR 8500
193	Khalid Mehmood	50	Male	03009862109	PKR 21000
194	Uzma Rani	26	Female	03451222109	PKR 4500
195	Ejaz Ahmed	44	Male	03331222109	PKR 16500
196	Naseem Akhtar	57	Female	03001221098	PKR 23000
197	Liaquat Ali	31	Male	03211221098	PKR 6500
198	Rubina Parveen	45	Female	03009861098	PKR 13500
199	Asif Mehmood	36	Male	03451221098	PKR 9500
200	Zakia Sultana	52	Female	03331221098	PKR 19500
201	Adnan Sami	27	Male	03001220987	PKR 5000
202	Bushra Ansari	41	Female	03211220987	PKR 10000

203	Imtiaz Ahmed	48	Male	03009860987	PKR 18000
204	Shehnaz Akhtar	34	Female	03451220987	PKR 7000
205	Ghazanfar Ali	39	Male	03331220987	PKR 12500
206	Yasmeen Bibi	23	Female	03001219876	PKR 3500
207	Mohsin Raza	55	Male	03211219876	PKR 24000
208	Parveen Sultana	60	Female	03009859876	PKR 29000
209	Aamir Liaquat	43	Male	03451219876	PKR 15000
210	Shirin Farhad	30	Female	03331219876	PKR 6500
211	Zaheer Abbas	37	Male	03001218765	PKR 10500
212	Fareeda Bibi	29	Female	03211218765	PKR 4500
213	Saqlain Mushtaq	46	Male	03009858765	PKR 17000
214	Kausar Parveen	33	Female	03451218765	PKR 8000
215	Mehmood Khan	61	Male	03331218765	PKR 33000
216	Rubab Zaidi	24	Female	03001217654	PKR 3000
217	Furrukh Sher	52	Male	03211217654	PKR 22000
218	Misbah ul Haq	49	Male	03009857654	PKR 20000
219	Iqra Aziz	22	Female	03451217654	PKR 2500
220	Rizwan Ahmed	35	Male	03331217654	PKR 9000
222	Tariq Hussain	38	Male	03001234569	PKR 6000
225	Khalid Anwar	55	Male	03451234561	PKR 18000
226	Nazia Iqbal	33	Female	03331234562	PKR 7500
227	Zahida Parveen	41	Female	03001216543	PKR 11000
228	Murad Ali	26	Male	03211216543	PKR 4000
229	Tehmina Durrani	50	Female	03009856543	PKR 19500
230	Saleem Akhtar	44	Male	03451216543	PKR 14000
231	Ghazala Javed	31	Female	03331216543	PKR 6000
232	Rao Anwar	58	Male	03001215432	PKR 26000
233	Perveen Shakir	36	Female	03211215432	PKR 8500
234	Asghar Khan	47	Male	03009855432	PKR 17500
235	Farzana Bibi	28	Female	03451215432	PKR 4500
236	Shaheen Baig	53	Male	03331215432	PKR 23000
237	Naseem Bibi	39	Female	03001214321	PKR 10000
238	Iftikhar Ahmed	42	Male	03211214321	PKR 14500
239	Saima Naz	25	Female	03009854321	PKR 3500
240	Zubair Khan	61	Male	03451214321	PKR 31000

241 Rauf Baig 0 Male 03001234599 PKR 4000
Total: 134 patient(s).

--- Patient Management ---

1. Add Patient
2. Update Patient
3. Delete Patient
4. View All Patients
0. Back

Choice: 0

===== Main Menu =====

1. Patient Management
2. Doctor Management
3. Appointments
4. Treatments & Billing
5. Search / Reports
6. Exit

=====

Enter your choice: 2

--- Doctor Management ---

1. Add Doctor
2. Update Doctor
3. Delete Doctor
4. View All Doctors
0. Back

Choice: 1

--- Add New Doctor ---

Doctor ID: 400

Name: Dr.Sheikh

Specialty: Leg checkup

Years of Experience: 15

Doctor added successfully.

--- Doctor Management ---

1. Add Doctor
2. Update Doctor
3. Delete Doctor
4. View All Doctors
0. Back

Choice: 2

Enter Doctor ID to update: 229

Current Name: Dr. Amna Siddiqui New Name (Enter to keep): Siddiqui Khan

Current Specialty: Geriatrics New Specialty (Enter to keep): Urologist

Current Experience: 13 yrs New Experience (0 to keep): 12

Doctor updated successfully.

--- Doctor Management ---

1. Add Doctor
2. Update Doctor
3. Delete Doctor
4. View All Doctors
0. Back

Choice: 3

Enter Doctor ID to delete: 228

Doctor deleted.

--- Doctor Management ---

1. Add Doctor
2. Update Doctor
3. Delete Doctor
4. View All Doctors
0. Back

Choice: 4

ID	Name	Specialty	Exp
201	Dr. Kamran Yousaf	Cardiologist	10 yrs
202	Dr. Sana Mirza	ENT	5 yrs
203	Dr. Imran Malik	Neurologist	8 yrs
204	Dr. Ayesha Farooq	Pediatrics	12 yrs
205	Dr. Bilal Chaudhry	Orthopedics	7 yrs
206	Dr. Hina Akhtar	Gynecology	15 yrs
207	Dr. Tariq Shah	Dermatology	3 yrs
208	Dr. Nadia Hussain	General Surgery	9 yrs
209	Dr. Faisal Raza	Urology	6 yrs
210	Dr. Zara Butt	Ophthalmology	11 yrs
211	Dr. Asad Qureshi	Pulmonology	4 yrs
212	Dr. Maryam Baig	Psychiatry	13 yrs
213	Dr. Omer Siddiqui	Gastroenterology	2 yrs
214	Dr. Lubna Khan	Radiology	16 yrs
215	Dr. Waseem Anwar	Emergency Medicine	14 yrs
216	Dr. Shoaib Akhtar	Nephrology	8 yrs
217	Dr. Rabia Noor	Oncology	10 yrs
218	Dr. Zeeshan Malik	Endocrinology	6 yrs
219	Dr. Farah Hussain	Rheumatology	7 yrs
220	Dr. Adnan Malik	Hematology	9 yrs
221	Dr. Samina Raza	Infectious Disease	5 yrs
222	Dr. Khalid Mehmood	Plastic Surgery	12 yrs
223	Dr. Tahira Shah	Anesthesiology	11 yrs
224	Dr. Naveed Ahmed	Vascular Surgery	8 yrs
225	Dr. Bushra Fatima	Neonatology	6 yrs
226	Dr. Irfan Baig	Sports Medicine	4 yrs
227	Dr. Saima Khan	Allergy & Immunology	7 yrs
229	Siddiqui Khan	Urologist	12 yrs
230	Dr. Rizwan Akhtar	Trauma Surgery	15 yrs
231	Dr. Parveen Bibi	Obstetrics	10 yrs
232	Dr. Shahzaib Raza	Neurosurgery	8 yrs
233	Dr. Kiran Malik	Family Medicine	5 yrs

234	Dr. Ghulam Nabi	Internal Medicine	17 yrs
235	Dr. Uzma Fatima	Clinical Pathology	6 yrs
400	Dr. Sheikh	Leg checkup	15 yrs

Total: 35 doctor(s).

--- Doctor Management ---

1. Add Doctor
2. Update Doctor
3. Delete Doctor
4. View All Doctors
0. Back

Choice: 0

===== Main Menu =====

1. Patient Management
2. Doctor Management
3. Appointments
4. Treatments & Billing
5. Search / Reports
6. Exit

=====

Enter your choice: 3

--- Appointments ---

1. Schedule Appointment
2. Cancel Appointment
3. View All Appointments
4. Weekly Schedule
0. Back

Choice: 1

--- Schedule Appointment ---

Patient ID: 104

Doctor ID: 229

Date (MM-DD-YYYY): 2026-7-23

Time (e.g. 10:00 AM): 15:15 AM

Appointment scheduled: Patient 104 with Doctor 229 on 7-23-2026 at 03:15 PM

--- Appointments ---

1. Schedule Appointment
2. Cancel Appointment
3. View All Appointments
4. Weekly Schedule
0. Back

Choice: 2

Cancel Appointment

Patient ID: 104

Doctor ID: 229

Date (MM-DD-YYYY): 7-23-2026

Appointment not found.

--- Appointments ---

1. Schedule Appointment
2. Cancel Appointment
3. View All Appointments
4. Weekly Schedule
0. Back

Choice: 3

Pat ID	Doc ID	Date	Time
101	201	30-03-2026	10:00 AM
102	202	30-03-2026	11:00 AM
103	203	31-03-2026	09:00 AM
104	204	31-03-2026	02:00 AM
105	205	01-04-2026	10:30 AM
106	206	01-04-2026	03:00 AM
107	207	02-04-2026	05:57 PM
108	208	02-04-2026	01:00 AM

109	209	03-04-2026	09:30 AM
110	210	03-04-2026	04:00 AM
111	201	04-04-2026	10:00 AM
114	212	05-04-2026	03:30 AM
115	213	06-04-2026	09:00 AM
116	214	06-04-2026	06:35 PM
117	215	07-04-2026	01:00 AM
118	203	07-04-2026	02:00 AM
119	204	08-04-2026	11:00 AM
120	205	08-04-2026	04:30 AM
121	216	09-04-2026	09:00 AM
122	217	09-04-2026	10:00 AM
123	218	10-04-2026	11:30 AM
124	219	10-04-2026	02:00 AM
125	220	11-04-2026	03:30 AM
126	221	11-04-2026	10:00 AM
127	222	12-04-2026	01:25 PM
128	223	12-04-2026	09:30 AM
129	224	13-04-2026	01:00 AM
130	225	13-04-2026	04:00 AM
131	201	14-04-2026	10:30 AM
132	202	14-04-2026	11:30 AM
133	203	15-04-2026	09:00 AM
134	204	15-04-2026	02:30 AM
135	205	16-04-2026	03:00 AM
136	206	16-04-2026	10:00 AM
137	207	17-04-2026	11:00 AM
138	208	17-04-2026	01:30 AM
139	209	18-04-2026	09:30 AM
140	210	18-04-2026	04:30 AM
141	211	19-04-2026	02:11 PM
142	212	19-04-2026	11:30 AM
143	213	20-04-2026	02:00 AM
144	214	20-04-2026	03:30 AM
145	215	21-04-2026	09:00 AM

146	216	21-04-2026	10:30 AM
147	217	22-04-2026	09:49 PM
148	218	22-04-2026	02:30 AM
149	219	23-04-2026	11:00 AM
150	220	23-04-2026	04:00 AM
151	221	04-24-2026	01:00 PM
153	203	04-25-2026	09:00 AM
154	224	25-04-2026	03:30 PM
155	225	04-26-2026	08:30 AM
156	226	26-04-2026	04:00 PM
157	227	04-27-2026	01:45 PM
158	228	27-04-2026	10:00 AM
104	229	7-23-2026	03:15 PM

Total: 56 appointment(s).

--- Appointments ---

1. Schedule Appointment
2. Cancel Appointment
3. View All Appointments
4. Weekly Schedule
0. Back

Choice: 4

Enter Doctor ID for weekly schedule: 229

--- Weekly Schedule for Doctor 229 ---

Time	Appointments
------	--------------

09:00 AM	Free
10:00 AM	Free
11:00 AM	Free
12:00 PM	Free
01:00 PM	Free
02:00 PM	Free
03:00 PM	Free

```
--- Appointments ---
1. Schedule Appointment
2. Cancel Appointment
3. View All Appointments
4. Weekly Schedule
0. Back
```

Choice: 0

```
===== Main Menu =====
```

```
1. Patient Management
2. Doctor Management
3. Appointments
4. Treatments & Billing
5. Search / Reports
6. Exit
```

```
=====
```

Enter your choice: 4

```
--- Treatments & Billing ---
```

```
1. Add Treatment Record
2. View Treatments
3. Update Payment Status
4. Generate Bill
0. Back
```

Choice: 1

```
--- Add Treatment ---
```

Patient ID: 104

Treatment Description: Hand checkup

Cost (PKR): 560000

Is bill paid? (y/n): Y

Insufficient balance (Balance: PKR 3000, Cost: PKR 560000). Cannot mark as paid.

Treatment record added.

```
--- Treatments & Billing ---
```

1. Add Treatment Record
2. View Treatments
3. Update Payment Status
4. Generate Bill
0. Back

Choice: 2

Enter Patient ID (0 for all): 106

Pat ID	Description	Cost (PKR)	Status
106	Prenatal Checkup	4000	Paid

--- Treatments & Billing ---

1. Add Treatment Record
2. View Treatments
3. Update Payment Status
4. Generate Bill
0. Back

Choice: 3

Update Payment Status

Patient ID: 107

Treatment description: Skin Allergy

Already marked Paid.

No matching unpaid treatment found.

--- Treatments & Billing ---

1. Add Treatment Record
2. View Treatments
3. Update Payment Status
4. Generate Bill
0. Back

Choice: 4

Generate Bill for Patient ID: 104

=====

HOSPITAL BILL SUMMARY

=====

Patient: Fatima Malik (ID: 104)

Contact: 03451234567

Treatment	Cost	Status
Child Vaccination	1500	Paid
Hand checkup	560000	Unpaid

Total Cost: PKR 561500

Amount Paid: PKR 1500

Outstanding: PKR 560000

=====

--- Treatments & Billing ---

1. Add Treatment Record
2. View Treatments
3. Update Payment Status
4. Generate Bill
0. Back

Choice: 0

===== Main Menu =====

1. Patient Management
2. Doctor Management
3. Appointments
4. Treatments & Billing
5. Search / Reports
6. Exit

=====

Enter your choice: 5

--- Search / Reports ---

1. Search Patient (by ID or Name)
2. Search Doctor (by ID or Specialty)
3. View Treatments by Doctor
4. Sort Doctors by Experience
5. Remove Duplicate Patients (re-scan)
0. Back

Choice: 1

Search Patient by:

1. ID
2. Name

Choice: 1

Enter Patient ID: 108

ID	Name	Age	Gender	Contact	Balance
108	Zara Hussain	27	Female	03009871234	PKR 5500

--- Search / Reports ---

1. Search Patient (by ID or Name)
2. Search Doctor (by ID or Specialty)
3. View Treatments by Doctor
4. Sort Doctors by Experience
5. Remove Duplicate Patients (re-scan)
0. Back

Choice: 2

Search Doctor by:

1. ID
2. Specialty

Choice: 2

Choice: 2

ID	Name	Specialty	Exp
----	------	-----------	-----

Specialty : Radiologist

No doctors found for that specialty.

--- Search / Reports ---

1. Search Patient (by ID or Name)
2. Search Doctor (by ID or Specialty)
3. View Treatments by Doctor
4. Sort Doctors by Experience
5. Remove Duplicate Patients (re-scan)
0. Back

Choice: 2

Search Doctor by:

1. ID
2. Specialty

Choice: 2

ID	Name	Specialty	Exp
----	------	-----------	-----

Specialty : Emergency Medicine

215	Dr. Waseem Anwar	Emergency Medicine	14 yrs
-----	------------------	--------------------	--------

--- Search / Reports ---

1. Search Patient (by ID or Name)
2. Search Doctor (by ID or Specialty)
3. View Treatments by Doctor
4. Sort Doctors by Experience
5. Remove Duplicate Patients (re-scan)
0. Back

Choice: 3

Search treatments by Doctor ID: 215

Pat ID	Description	Cost	Status
117	Skin Biopsy	8000	Unpaid
145	Wisdom Tooth Extraction	6000	Paid

--- Search / Reports ---

1. Search Patient (by ID or Name)
2. Search Doctor (by ID or Specialty)
3. View Treatments by Doctor
4. Sort Doctors by Experience
5. Remove Duplicate Patients (re-scan)
0. Back

Choice: 4

Doctors sorted by experience (ascending):

ID	Name	Specialty	Exp
213	Dr. Omer Siddiqui	Gastroenterology	2 yrs
207	Dr. Tariq Shah	Dermatology	3 yrs
211	Dr. Asad Qureshi	Pulmonology	4 yrs
226	Dr. Irfan Baig	Sports Medicine	4 yrs
202	Dr. Sana Mirza	ENT	5 yrs
221	Dr. Samina Raza	Infectious Disease	5 yrs
233	Dr. Kiran Malik	Family Medicine	5 yrs
209	Dr. Faisal Raza	Urology	6 yrs
218	Dr. Zeeshan Malik	Endocrinology	6 yrs
225	Dr. Bushra Fatima	Neonatology	6 yrs
235	Dr. Uzma Fatima	Clinical Pathology	6 yrs
205	Dr. Bilal Chaudhry	Orthopedics	7 yrs
219	Dr. Farah Hussain	Rheumatology	7 yrs
227	Dr. Saima Khan	Allergy & Immunology	7 yrs

203	Dr. Imran Malik	Neurologist	8 yrs
216	Dr. Shoaib Akhtar	Nephrology	8 yrs
224	Dr. Naveed Ahmed	Vascular Surgery	8 yrs
232	Dr. Shahzaib Raza	Neurosurgery	8 yrs
208	Dr. Nadia Hussain	General Surgery	9 yrs
220	Dr. Adnan Malik	Hematology	9 yrs
201	Dr. Kamran Yousaf	Cardiologist	10 yrs
217	Dr. Rabia Noor	Oncology	10 yrs
231	Dr. Parveen Bibi	Obstetrics	10 yrs
210	Dr. Zara Butt	Ophthalmology	11 yrs
223	Dr. Tahira Shah	Anesthesiology	11 yrs
204	Dr. Ayesha Farooq	Pediatrics	12 yrs
222	Dr. Khalid Mehmood	Plastic Surgery	12 yrs
229	Siddiqui Khan	Urologist	12 yrs
212	Dr. Maryam Baig	Psychiatry	13 yrs
215	Dr. Waseem Anwar	Emergency Medicine	14 yrs
206	Dr. Hina Akhtar	Gynecology	15 yrs
230	Dr. Rizwan Akhtar	Trauma Surgery	15 yrs
400	Dr. Sheikh	Leg checkup	15 yrs
214	Dr. Lubna Khan	Radiology	16 yrs
234	Dr. Ghulam Nabi	Internal Medicine	17 yrs

--- Search / Reports ---

1. Search Patient (by ID or Name)
2. Search Doctor (by ID or Specialty)
3. View Treatments by Doctor
4. Sort Doctors by Experience
5. Remove Duplicate Patients (re-scan)
0. Back

Choice: 5

Duplicate check complete. Active records: 134

--- Search / Reports ---

1. Search Patient (by ID or Name)
2. Search Doctor (by ID or Specialty)
3. View Treatments by Doctor
4. Sort Doctors by Experience
5. Remove Duplicate Patients (re-scan)
0. Back

Choice: 0

===== Main Menu =====

1. Patient Management
2. Doctor Management
3. Appointments
4. Treatments & Billing
5. Search / Reports
6. Exit

=====

Enter your choice: